CHANDIGARH UNIVERSITY

Master of Computer Application (MCA)

PROJECT REPORT

On

CI/CD Pipeline Engineering for Kubernetes-Based Application Deployment

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Application (MCA)

Student Name	Akshit Kapil
Enrollment No.	O24MCA110895
Batch	MCA July 2024
Academic Year	2024-2026
GitHub Repository	https://github.com/zoro2024/CU-MCA-final-project.git

Under the Guidance of

(No mentor was assigned / Mentor did not conduct any session)

Chandigarh University, Punjab, India

2024-2026

Certificate

This is to certify that the project report titled:

“CI/CD Pipeline Engineering for Kubernetes-Based Application Deployment”

has been submitted by Akshit Kapil (Enrollment No.: O24MCA110895) in partial fulfillment of the requirements for the award of the degree of Master of Computer Application (MCA) at Chandigarh University Online, Punjab, during the academic year 2024–2026.

This project is a bonafide work carried out by the student. The content presented is original and has not been submitted to any other institution for the award of any degree or diploma.

Note: No Qollabb certificate has been received. The Qollabb platform certificate section is intentionally left blank pending receipt.

Note: No mentor was assigned for this project, and no mentorship sessions were conducted. The mentor did not reply on the Qollabb platform chat. This project has been completed independently by the student.

Declaration

I, Akshit Kapil, hereby solemnly declare that the project report titled “CI/CD Pipeline Engineering for Kubernetes-Based Application Deployment” submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Application (MCA) is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2024–2026 independently, as no mentor was assigned and no mentorship sessions were conducted.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: Meerut, Uttar Pradesh

Date: 28 May 2026

Student Signature: *AKSHIT KAPIL*

Student Name: Akshit Kapil

Enrollment No.: O24MCA110895

Acknowledgement

I express my sincere gratitude to Chandigarh University Online for providing the platform and resources to undertake this project as part of the MCA curriculum.

I acknowledge the supportive environment provided by the university's academic framework, which allowed me to independently design, develop, and document a production-grade CI/CD pipeline system based on modern cloud-native technologies.

I am grateful to the open-source communities behind Jenkins, ArgoCD, Argo Rollouts, Kubernetes, Helm, Docker, SonarQube, Trivy, and the AWS ecosystem, whose tools and documentation formed the technical backbone of this project.

Special thanks to Google for providing the Gemini AI API which powers the intelligent log analysis feature of the LogLens application deployed through this pipeline.

This project was completed independently, which, was challenging, has greatly strengthened my understanding of DevOps engineering, infrastructure automation, and cloud-native application delivery.

Abstract

This project presents the end-to-end engineering of a CI/CD (Continuous Integration and Continuous Delivery) pipeline for deploying a containerized, cloud-native application — LogLens AI — on a Kubernetes cluster hosted on Amazon EKS. LogLens AI is an intelligent log analysis application built with a FastAPI backend, a React/Vite frontend, PostgreSQL for persistent storage, and Redis for caching, with AI-powered log analysis via the Google Gemini API.

The CI/CD pipeline automates the complete software delivery lifecycle: source code is pushed to GitHub, Jenkins triggers automated builds incorporating security scanning (Gitleaks for secret detection, SonarQube for static code analysis, Trivy for container vulnerability scanning), Docker images are built and pushed to AWS ECR, Helm chart values are updated programmatically, and ArgoCD synchronizes the desired state into the Kubernetes cluster using GitOps principles. Progressive delivery is achieved via Argo Rollouts implementing canary deployment strategies, enabling zero-downtime releases with automatic rollback on failure.

The system architecture covers the full spectrum of production-readiness: NGINX ingress for traffic routing via AWS ALB, Horizontal Pod Autoscaling (HPA) for workload elasticity, Prometheus metrics instrumentation, and Kubernetes Secret management for sensitive configuration. The project demonstrates practical application of DevOps best practices including infrastructure-as-code, GitOps, shift-left security, and declarative continuous delivery in a real production-grade environment.

Project Title	CI/CD Pipeline Engineering for Kubernetes-Based Application Deployment
Application	LogLens AI — AI-powered Log Analysis System
Technology Stack	Jenkins, ArgoCD, Argo Rollouts, Docker, AWS ECR, AWS EKS, Helm, Kubernetes, SonarQube, Trivy, Gitleaks
Programming Languages	Python (FastAPI), JavaScript (React/Vite), Groovy (Jenkinsfile), YAML
Repository	https://github.com/zoro2024/CU-MCA-final-project/tree/master

Chapter 1: Introduction

1.1 Background and Motivation

The rapid evolution of software engineering practices over the past decade has led to a paradigm shift from monolithic, manually-deployed applications to containerized, microservices-based systems deployed through fully automated pipelines. Modern software teams increasingly rely on DevOps philosophies — a cultural and technical movement that bridges development and operations — to accelerate delivery cycles while maintaining quality and reliability.

Kubernetes has emerged as the de facto standard for container orchestration, providing declarative configuration, self-healing, horizontal scalability, and a rich ecosystem of tooling. However, operating a Kubernetes-based application requires significantly more than just writing YAML manifests; it demands robust automation for building, testing, scanning, packaging, and deploying software artifacts in a repeatable and auditable manner.

CI/CD pipelines serve as the automation backbone of modern software delivery. A well-engineered pipeline reduces human error, enforces code quality gates, integrates security scanning at every stage (shifting security left), and ensures that only validated, verified artifacts reach production environments. The adoption of GitOps — where the Git repository is the single source of truth for infrastructure and application state — further strengthens auditability and enables automated reconciliation of the desired versus actual cluster state.

1.2 Problem Statement

Organizations deploying containerized applications on Kubernetes frequently encounter the following challenges:

- Manual, error-prone deployment processes that lack consistency across environments.
- Absence of security scanning in the delivery pipeline, resulting in vulnerable images reaching production.
- Lack of progressive delivery mechanisms, forcing all-or-nothing releases that risk complete service outages on bad deployments.
- Poor observability into deployment status, with no automated rollback when deployments fail health checks.
- Disconnected tooling with no unified pipeline orchestrating build, test, scan, and deploy stages.

This project addresses these challenges by designing and implementing a comprehensive, production-grade CI/CD pipeline that integrates all stages of secure software delivery for a Kubernetes-based application.

1.3 Objectives

1. Design and implement a Jenkins-based CI/CD pipeline that automates the full software delivery lifecycle from code commit to production deployment on Kubernetes.
2. Integrate multi-layered security scanning: Gitleaks (secret detection), SonarQube (static code analysis), and Trivy (container image vulnerability scanning).
3. Implement GitOps-based continuous delivery using ArgoCD, making the Git repository the authoritative source of truth for cluster state.
4. Enable progressive delivery through Argo Rollouts with canary deployment strategy and automated rollback capabilities.
5. Deploy the LogLens AI application — a FastAPI + React application with PostgreSQL, Redis, and Google Gemini AI integration — as the target workload.
6. Package all Kubernetes resources using Helm charts for templated, reusable, and version-controlled deployments.
7. Expose application services through AWS ALB Ingress Controller for production-grade HTTP/HTTPS routing.
8. Enable autoscaling through Horizontal Pod Autoscaler (HPA) and integrate Prometheus metrics instrumentation.

1.4 Scope of the Project

The project encompasses:

- Development of a sample AI-powered web application (LogLens AI) as the deployment target.
- Complete Dockerization of frontend (React/Nginx) and backend (FastAPI) services.
- Construction of parameterized Jenkins pipelines (Canary Rollout and Rollout Existing Tag modes) for both backend and frontend services.
- Configuration and deployment of ArgoCD for GitOps-driven continuous delivery.
- Installation and configuration of Argo Rollouts for canary progressive delivery on Amazon EKS.
- Setup of AWS infrastructure including EKS, ECR, ALB Ingress Controller, and EBS CSI Driver for persistent storage.
- Deployment of supporting services: PostgreSQL (Bitnami Helm chart with EBS persistence), Redis (Bitnami Helm chart), and SonarQube for code quality analysis.
- Helm chart development for both backend and frontend including deployments, services, ingress, HPA, and ConfigMaps.

1.5 Organization of the Report

This report is organized as follows:

- Chapter 2 provides a System Study covering existing tools, technologies, and literature survey.
- Chapter 3 presents System Analysis including requirements specification and feasibility study.
- Chapter 4 details System Design covering architecture, pipeline design, and component design.
- Chapter 5 covers System Implementation with code, configuration, and deployment details.
- Chapter 6 describes the Testing strategy and results.
- Chapter 7 presents Results and Discussion.
- Chapter 8 concludes with summary and future scope.
- Chapter 9 lists references and Chapter 10 contains appendices.

Chapter 2: System Study

2.1 Study of Existing Systems

Before designing this project, an extensive study of existing CI/CD and Kubernetes deployment solutions was conducted.

2.1.1 Traditional Deployment Models

Traditional software deployments involved manual steps: developers would build artifacts on local machines, SCP them to remote servers, and restart services manually. These approaches lacked repeatability, audit trails, and rollback capabilities. Human errors were common, and deployment downtime was frequent.

2.1.2 Basic CI/CD with Jenkins

Jenkins is one of the oldest and most widely adopted automation servers in the industry. Traditional Jenkins usage involved Freestyle jobs that ran simple build scripts. While functional, these approaches lacked pipeline-as-code capabilities, were difficult to version control, and had no native Kubernetes integration. Modern Jenkins usage leverages declarative pipelines (Jenkinsfile) that define the entire build-deploy workflow in code, stored in the same repository as the application.

2.1.3 Container-Based Deployments

The introduction of Docker revolutionized software packaging by providing lightweight, isolated containers that encapsulate an application and all its dependencies. Docker Compose enabled multi-container local development environments. However, Docker alone does not provide production-grade features like automated scheduling, self-healing, rolling updates, or resource management — necessitating a container orchestration platform.

2.1.4 Kubernetes as Orchestration Platform

Kubernetes (K8s) addresses the orchestration gap, providing declarative configuration of desired state, automated bin-packing, self-healing through liveness/readiness probes, horizontal scaling, rolling updates, and a rich plugin ecosystem. Amazon EKS (Elastic Kubernetes Service) provides a managed Kubernetes control plane on AWS, eliminating the operational overhead of managing etcd, API servers, and controller managers.

2.1.5 GitOps with ArgoCD

GitOps is a paradigm in which the entire system state — application configuration, Kubernetes manifests, Helm values — is stored in a Git repository. A GitOps controller (ArgoCD in this project) continuously monitors the repository and reconciles the actual cluster state to match the desired state

declared in Git. This provides auditability, pull-based deployment (eliminating the need for CI systems to push directly to the cluster), and natural disaster recovery through Git history.

2.1.6 Progressive Delivery

Traditional Kubernetes rolling updates replace all pods simultaneously or in batches, which can still cause widespread outages if a bad release is deployed. Argo Rollouts extends Kubernetes deployments with canary and blue-green strategies. Canary deployments gradually shift a percentage of traffic to the new version, monitor metrics, and promote or abort the rollout based on health checks — significantly reducing the blast radius of bad releases.

2.2 Literature Survey

Several key concepts and tools were studied during the preparation of this project:

- Humble & Farley (2010), Continuous Delivery — introduced the principles of deployment pipelines and the importance of automating the path to production.
- Fowler (2006), Continuous Integration — established the foundational practice of integrating code frequently with automated builds and tests.
- Weaveworks GitOps principles (2017) — formalized GitOps as an operational model for cloud-native applications.
- NIST SP 800-190 (Application Container Security Guide) — informed the security scanning strategy for container images and secrets management.
- CIS Kubernetes Benchmark — guided hardening decisions for the Kubernetes cluster configuration.
- AWS Well-Architected Framework (DevOps Lens) — provided best practices for operational excellence, security, reliability, and cost optimization on AWS.

2.3 Technology Stack Overview

Category	Tool / Technology	Purpose
CI/CD Orchestration	**Jenkins (Declarative Pipeline)**	Automate build, test, scan, and deploy stages
GitOps / CD	**ArgoCD**	Continuous delivery via Git repository sync
Progressive Delivery	**Argo Rollouts**	Canary deployments with automated rollback
Container Registry	**AWS ECR**	Store and version Docker images
Container Orchestration	**Amazon EKS (Kubernetes)**	Run containerized workloads at scale
Package Manager	**Helm**	Kubernetes manifest templating and versioning
Ingress Controller	**AWS ALB Controller**	HTTP/HTTPS traffic routing
Secret Detection	**Gitleaks**	Detect hardcoded secrets in source code
Code Quality	**SonarQube**	Static analysis, bug detection, coverage
Image Scanning	**Trivy**	Detect CVEs in Docker images
Backend Framework	**FastAPI (Python)**	REST API with async support
Frontend Framework	**React + Vite + Nginx**	SPA served via production Nginx

Database	**PostgreSQL (Bitnami/Helm)**	Persistent relational storage
Cache	**Redis (Bitnami/Helm)**	API response caching
AI Integration	**Google Gemini API**	Intelligent log analysis
Storage	**AWS EBS CSI Driver**	Persistent volumes for DB/cache
Autoscaling	**Kubernetes HPA**	Horizontal pod autoscaling
Monitoring	**Prometheus (FastAPI Instrumentator)**	Metrics instrumentation

Chapter 3: System Analysis

3.1 Requirements Analysis

3.1.1 Functional Requirements

1. The system shall accept a Git commit push event as a trigger for the CI/CD pipeline.
2. The pipeline shall perform automated security scanning including secret detection, static code analysis, and container vulnerability scanning at every build.
3. The pipeline shall build Docker images and push them to AWS ECR with a tag derived from the Git commit hash and Jenkins build number.
4. The pipeline shall update Helm chart values files with the new image tag and commit the change back to the Git repository.
5. ArgoCD shall automatically detect the Helm values update and synchronize the cluster state.
6. The pipeline shall support two deployment modes: Canary Rollout (full build pipeline) and Rollout Existing Tag (reuse an existing image tag, skip build).
7. On deployment failure, the pipeline shall automatically revert the Helm values commit and re-sync ArgoCD to trigger rollback.
8. The backend API shall expose a /healthz endpoint, /metrics endpoint, and /api/* REST endpoints.
9. The application shall accept raw log text as input and return a structured AI-generated analysis report via the Gemini API.
10. All reports shall be persisted in PostgreSQL and retrievable via GET endpoints.

3.1.2 Non-Functional Requirements

- Availability: The deployment strategy (canary) shall ensure zero downtime during upgrades.
- Security: No hardcoded secrets shall exist in any code file. All sensitive values shall be stored in Kubernetes Secrets.
- Scalability: The system shall support horizontal scaling of both backend and frontend pods through HPA.
- Maintainability: All configuration shall be as-code (Jenkinsfile, Helm charts, Kubernetes manifests) and version-controlled in Git.
- Auditability: Every deployment change shall be traceable to a Git commit, providing a full audit trail.
- Performance: API response times for cached queries shall be under 100ms. Uncached AI analysis responses depend on Gemini API latency.
- Recoverability: Automated rollback shall restore the previous stable version within minutes of a detected deployment failure.

3.2 Feasibility Study

3.2.1 Technical Feasibility

All tools used in this project are production-proven, open-source (or free tier available), and well-documented. Jenkins, ArgoCD, Argo Rollouts, Helm, and Kubernetes are widely adopted at enterprise scale. AWS EKS provides a managed Kubernetes control plane eliminating infrastructure complexity. The technology choices are technically feasible and have been validated through real implementation and deployment in this project.

3.2.2 Economic Feasibility

The project uses a combination of open-source tools (Jenkins, ArgoCD, Helm, Argo Rollouts, SonarQube, Trivy, Gitleaks) and AWS free-tier / pay-as-you-go resources. The Google Gemini API offers a generous free tier suitable for development and academic use. The total cost of running this project for development and demonstration purposes is minimal, making it economically feasible.

3.2.3 Operational Feasibility

The system is designed for operation by a DevOps or platform engineering team with standard cloud infrastructure skills. The GitOps approach reduces operator burden by enabling self-healing reconciliation. Helm charts package all deployment complexity into manageable, reusable templates. Detailed README documentation in the repository ensures that the system can be understood, operated, and extended by any technically proficient engineer.

3.3 System Architecture Overview

The high-level system architecture follows a GitOps flow:

1. Developer pushes code to GitHub repository.
2. Jenkins (triggered via webhook or manual run) pulls the latest code.
3. Jenkins runs Gitleaks to scan for hardcoded secrets in the repository.
4. Jenkins runs SonarQube analysis for static code quality and security issues.
5. Jenkins builds the Docker image for the changed service (backend or frontend).
6. Jenkins runs Trivy to scan the freshly built Docker image for known CVEs.
7. Jenkins pushes the verified Docker image to AWS ECR with a unique tag.
8. Jenkins updates the image tag in the Helm values.yaml and pushes the change to GitHub.
9. ArgoCD detects the updated values.yaml and triggers a sync against the EKS cluster.
10. Argo Rollouts progressively shifts traffic to the new version using canary strategy.
11. On failure, the pipeline reverts the Git commit and ArgoCD rolls back the cluster.

Chapter 4: System Design

4.1 Overall Pipeline Design

The pipeline is organized into two parameterized modes accessible through the Jenkins UI:

- Canary Rollout: Full pipeline — checkout → Gitleaks → SonarQube → ECR login → Docker build → Trivy scan → ECR push → Helm values update → Git push → ArgoCD sync → health wait.
- Rollout Existing Tag: Lightweight mode — uses an existing ECR image tag, skips build stages → updates Helm values → ArgoCD sync. Useful for promoting a previously validated image across environments.

The pipeline is implemented as a Groovy declarative Jenkinsfile stored in the repository, one per service (Backend/Jenkinsfile, Frontend/Jenkinsfile).

4.2 Jenkins Pipeline Stage Design

#	Stage	Description
1	**Checkout Code**	Clones the specified Git branch from the GitHub repository.
2	**Set Image Variables**	Generates unique image tag from Git short commit hash + Jenkins build number (e.g., a1b2c3d-42).
3	**Gitleaks Scan**	Scans codebase for hardcoded secrets, API keys, and tokens. Generates gitleaks-report.json.
4	**SonarQube Analysis**	Runs static analysis via SonarQube Scanner. Checks code quality gates before proceeding.
5	**ECR Login**	Authenticates Docker daemon with AWS ECR using aws ecr get-login-password.
6	**Docker Build**	Builds Docker image from the service Dockerfile with the generated tag.
7	**Trivy Image Scan**	Scans the built Docker image for OS and package CVEs. Generates trivy-report.html.
8	**Push to ECR**	Pushes the scanned, verified image to AWS ECR repository.
9	**Update Helm Values**	Updates image tag in Application/helm/<service>/values.yaml via sed/yq.
10	**Git Push**	Commits and pushes the updated values.yaml to GitHub repository.
11	**ArgoCD Sync**	Logs into ArgoCD CLI and triggers application sync, deploying the new image to Kubernetes.
12	**Wait for Health**	Polls ArgoCD application health until Healthy/Synced or timeout.

		Triggers rollback on failure.
13	**Auto Rollback**	On failure: reverts the Git commit and triggers ArgoCD re-sync to restore previous state.

4.3 Application Architecture Design

4.3.1 LogLens AI Application

LogLens AI is the target workload deployed through the CI/CD pipeline. It is a multi-tier application:

- Frontend: React 18 + Vite SPA, built as static assets and served by an Nginx container. Nginx also acts as a reverse proxy, forwarding /api/*, /healthz, and /metrics requests to the backend service.
- Backend: FastAPI (Python) REST API providing log analysis endpoints (/api/logs/analyze, /api/reports), health check (/healthz), and Prometheus metrics (/metrics). Uses SQLAlchemy ORM for PostgreSQL access and a Redis cache layer for API response caching.
- AI Layer: backend/app/ai.py integrates Google Gemini API via the OpenAI-compatible SDK. Accepts raw log text, truncates if necessary, and returns structured JSON analysis (severity, summary, error counts, root causes, suggested fixes, confidence score).
- Database: PostgreSQL stores AnalysisReport and AnalysisFinding records persistently. Deployed via Bitnami Helm chart with AWS EBS gp3 persistent volumes.
- Cache: Redis stores serialized analysis results keyed by log content hash, reducing redundant AI API calls and improving response times.

4.3.2 Kubernetes Resource Design

Each service (backend, frontend) is deployed with the following Kubernetes resources:

- Argo Rollout (instead of Deployment): defines canary strategy with configurable step weights and pause durations.
- Service: ClusterIP service exposing the pod ports within the cluster.
- Ingress: AWS ALB Ingress with appropriate annotations for Internet-facing load balancer, target type, and health check paths.
- HPA (Horizontal Pod Autoscaler): scales pods based on CPU utilization metric, with configurable min/max replicas.
- ConfigMap: stores non-sensitive application configuration (e.g., Nginx configuration for the frontend).
- Kubernetes Secret (loglens-secrets): stores GEMINI_API_KEY, DATABASE_URL, and REDIS_URL. Created externally (not by Helm) for production security.

4.4 Helm Chart Design

Helm charts are developed for both backend and frontend services following the standard Helm chart structure:

- Chart.yaml: defines chart name, version, and description.
- values.yaml: provides parameterized defaults for image repository, image tag, ingress host, HPA settings, and resource limits.
- templates/rollout.yaml: Argo Rollout resource with canary steps templated from values.
- templates/service.yaml: Kubernetes Service resource.
- templates/ingress.yaml: ALB Ingress resource with annotations.
- templates/hpa.yaml: Horizontal Pod Autoscaler resource.
- templates/configmap.yaml: ConfigMap for frontend Nginx configuration.
- templates/job.yaml (backend only): Helm hook Job to run database initialization on install/upgrade.

4.5 Security Design

Security is embedded at every layer of the pipeline and application:

- Pre-commit layer: Gitleaks scans every build for secrets and credentials committed by mistake.
- Code quality layer: SonarQube enforces quality gates including security hotspot review, blocking pipeline progression on critical issues.
- Image layer: Trivy scans Docker images for OS package CVEs and application dependency vulnerabilities before push to ECR.
- Runtime layer: Kubernetes Secrets store sensitive values. No secrets appear in Helm values files, Dockerfiles, or application code.
- Network layer: AWS ALB provides TLS termination. Nginx enforces path-based routing, preventing direct access to backend internals.
- IAM layer: AWS IRSA (IAM Roles for Service Accounts) provides the ALB Controller with least-privilege AWS permissions without static credentials.

Chapter 5: System Implementation

5.1 Infrastructure Setup

5.1.1 Amazon EKS Cluster

An Amazon EKS cluster was provisioned as the Kubernetes runtime environment. The cluster uses managed node groups for worker nodes, and IAM OIDC provider integration is configured to enable IRSA (IAM Roles for Service Accounts) for the AWS Load Balancer Controller.

5.1.2 AWS Load Balancer Controller (Ingress Controller)

The AWS Load Balancer Controller was installed on EKS using Helm, configured with an IAM service account that has permissions to create and manage ALBs. This controller watches Kubernetes Ingress resources and provisions AWS Application Load Balancers automatically.

Key setup steps:

1. Associate IAM OIDC provider with the EKS cluster.
2. Create IAM policy for the Load Balancer Controller from the official AWS policy JSON.
3. Create IAM role with OIDC trust relationship for the aws-load-balancer-controller service account.
4. Install the controller via Helm:

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \

  -n kube-system \

  --set clusterName=<cluster-name> \

  --set serviceAccount.create=false \

  --set serviceAccount.name=aws-load-balancer-controller
```

5.1.3 PostgreSQL Deployment

PostgreSQL was deployed using the Bitnami Helm chart in the postgres namespace, backed by AWS EBS gp3 persistent volumes provisioned via the EBS CSI Driver:

```
helm install my-postgres bitnami/postgresql \

  --namespace postgres --create-namespace \

  -f Tools-installation/postgres/values.yaml \

  --set-file global.postgresql.auth.existingSecret=Tools-installation/postgres/secret.yaml
```

5.1.4 Redis Deployment

Redis was deployed in standalone mode (authentication disabled for simplicity) using the Bitnami Helm chart, also backed by AWS EBS gp3 volumes:

```
helm install my-redis bitnami/redis \

  --namespace redis --create-namespace \

  --set auth.enabled=false \

  --set master.persistence.storageClass=gp3
```

5.1.5 ArgoCD Deployment

ArgoCD was installed in the argocd namespace from the official manifests, then configured with insecure mode (TLS terminated at ALB) and exposed via an ALB Ingress. Argo Rollouts was installed separately in the argo-rollouts namespace with its dashboard.

```
kubectl create namespace argocd

kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

kubectl patch configmap argocd-cmd-params-cm -n argocd --type merge -p '{"data": {"server.insecure":"true"}}'
```

5.2 Application Implementation

5.2.1 Backend (FastAPI)

The FastAPI backend is implemented in Python with the following modules:

- app/main.py: defines FastAPI application, middleware (CORS, Prometheus), startup event (DB init), and all API route handlers.
- app/ai.py: implements the AI integration with Google Gemini via the OpenAI-compatible SDK. Uses tool_choice to force structured JSON output conforming to the Analysis TypedDict schema.
- app/db.py: defines SQLAlchemy models (AnalysisReport, AnalysisFinding) and database session management.
- app/cache.py: implements Redis-based response caching using a hash of the log content as the cache key.
- app/config.py: pydantic-settings based configuration loading from environment variables.
- app/schemas.py: Pydantic schemas for request/response validation (AnalyzeRequest, ReportOut, ReportFull).

5.2.2 Frontend (React + Vite)

The React frontend is built with Vite and styled with Tailwind CSS. It provides:

- A log input form where users paste raw log text and submit for analysis.

- Real-time display of the structured analysis result including severity score, summary, error counts, root causes, and suggested fixes.
- A reports list view that fetches and displays historical analysis reports from the backend.
- A 3D background effect implemented in background3d.js using Three.js for visual polish.

In production, the frontend is containerized in an Nginx Docker image. The nginx.conf proxies /api/*, /healthz, and /metrics to the backend service, and serves /frontend-healthz for health checking.

5.2.3 Docker Implementation

Each service has a dedicated Dockerfile:

- Backend Dockerfile: multi-stage build using Python slim image, installs dependencies from requirements.txt, copies app code, exposes port 8000.
- Frontend Dockerfile: multi-stage build — Node 18 Alpine stage runs npm ci and vite build, Nginx Alpine stage copies built assets and nginx.conf.

5.3 Jenkins Pipeline Implementation

5.3.1 Pipeline Environment Configuration

The pipeline is parameterized with the following environment variables defined in the Jenkinsfile:

```
environment {

    APP_NAME        = "loglens-backend"

    AWS_REGION      = "ap-south-1"

    AWS_ACCOUNT_ID  = "432995435807"

    ECR_REPO        = "cu-final/${APP_NAME}"

    REGISTRY        = "${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"

    ECR_URI         = "${REGISTRY}/${ECR_REPO}"

    APP_PATH        = "Application/backend"

    HELM_PATH       = "Application/helm/backend"

    GIT_REPO        = "https://github.com/zoro2024/CU-MCA-final-project.git"

    ARGOCD_APP_NAME = "loglens-backend"

}
```

5.3.2 Security Scanning Implementation

Gitleaks scan stage:

```
stage('Gitleaks Scan') {
```

```
    steps {
        sh 'gitleaks detect --source . --report-path gitleaks-report.json'
    }
}
```

Trivy image scan stage:

```
stage('Trivy Scan') {
    steps {
        sh '''trivy image --format template \
              --template @/usr/local/share/trivy/templates/html.tpl \
              -o trivy-report.html ${ECR_URI}:${IMAGE_TAG}'''
    }
}
```

5.3.3 Automatic Rollback Implementation

On deployment failure, the pipeline executes the rollback post stage:

```
post {
    failure {
        script {
            sh 'git revert HEAD --no-edit'
            sh 'git push origin master'
            sh 'argocd app sync ${ARGOCD_APP_NAME}'
        }
    }
}
```

5.4 Argo Rollout (Canary) Implementation

The Argo Rollout resource replaces the standard Kubernetes Deployment. The canary strategy is defined in the Helm chart template:

```
strategy:
  canary:
    steps:
      - setWeight: 20
      - pause: { duration: 30s }
      - setWeight: 50
      - pause: { duration: 30s }
      - setWeight: 100
```

This progressively shifts 20% → 50% → 100% of traffic to the new version, pausing between steps to allow health metrics to stabilize. If Argo Rollouts detects failures, it can abort and shift traffic back to the stable version.

5.5 ArgoCD Application Configuration

ArgoCD applications are configured to watch the GitHub repository and synchronize Helm releases to the EKS cluster. The ArgoCD application spec points to the Helm chart path and the target Kubernetes namespace, with automated sync and self-heal policies appropriate for the production environment.

Chapter 6: Testing

6.1 Testing Strategy

Testing for a CI/CD pipeline project covers multiple layers: unit tests for application logic, integration tests for API endpoints, infrastructure validation for Kubernetes resources, and end-to-end pipeline execution tests.

6.2 Application Testing

6.2.1 Backend API Testing

Test Case	Endpoint	Expected Result	Status
Health Check	`GET /healthz`	HTTP 200 {status: ok, redis: true}	**PASS**
Log Analysis	`POST /api/logs/analyze`	HTTP 200 with structured analysis JSON	**PASS**
List Reports	`GET /api/reports`	HTTP 200 with array of report objects	**PASS**
Get Report by ID	`GET /api/reports/{id}`	HTTP 200 with full report including raw_log	**PASS**
Report Not Found	`GET /api/reports/invalid-id`	HTTP 404 Report not found	**PASS**
Metrics Endpoint	`GET /metrics`	HTTP 200 with Prometheus metrics	**PASS**
Cache Hit	`POST /api/logs/analyze (same input twice)`	Second call returns cached result faster	**PASS**
Frontend Health	`GET /frontend-healthz`	HTTP 200 from Nginx	**PASS**

6.3 Pipeline Testing

6.3.1 Canary Rollout Pipeline Test

A complete end-to-end Canary Rollout pipeline run was executed and validated:

1. Code change committed and pushed to GitHub master branch.
2. Jenkins pipeline triggered, executing all stages in sequence.

3. Gitleaks scan completed with no secret findings.
4. SonarQube analysis passed quality gate.
5. Docker image built successfully with tag <commit-hash>-<build-number>.
6. Trivy scan completed, generating HTML vulnerability report.
7. Image pushed to AWS ECR repository.
8. Helm values.yaml updated with new image tag and committed to Git.
9. ArgoCD detected the change and initiated sync.
10. Argo Rollouts executed canary steps: 20% → 30s pause → 50% → 30s pause → 100%.
11. Deployment reached Healthy/Synced state. Pipeline marked SUCCESS.

6.3.2 Rollback Testing

To validate the rollback mechanism, a deliberately broken image tag was introduced:

1. A deployment was triggered with a bad configuration that caused pods to fail readiness probes.
2. ArgoCD reported the application as Degraded after the health check timeout.
3. Jenkins post-failure hook triggered: Git commit reverted, ArgoCD re-sync executed.
4. Previous stable image tag restored to Helm values and pushed to Git.
5. ArgoCD synced the reverted values, Argo Rollouts deployed the stable image.
6. Application returned to Healthy state.

6.4 Security Scanning Test Results

- Gitleaks: Scanned 100% of repository files. Zero secret leaks detected in project codebase.
- SonarQube: Static analysis completed. Backend Python code achieved 0 critical issues. Frontend JavaScript passed quality gate.
- Trivy: Docker image scan completed. Known critical CVEs from base images logged in HTML report for review. Pipeline configured to report without blocking on informational findings (configurable threshold).

6.5 Scalability Testing

The Horizontal Pod Autoscaler was validated by artificially loading the backend API:

1. HPA configured with minReplicas=1, maxReplicas=5, targetCPUUtilizationPercentage=70.
2. Load applied using curl in a loop simulating concurrent requests.
3. HPA scaled backend from 1 pod to 3 pods within 2 minutes as CPU exceeded 70%.
4. Load removed; HPA scaled back to 1 pod after stabilization period.

Chapter 7: Results & Discussion

7.1 Project Outcomes

The project successfully achieved all stated objectives. A production-grade, fully automated CI/CD pipeline was designed, implemented, and validated for the LogLens AI application deployed on Amazon EKS.

7.1.1 Pipeline Performance

Metric	Observed Value
Average full pipeline duration (Canary Rollout)	**~8–12 minutes**
Average lightweight pipeline duration (Rollout Existing Tag)	**~3–4 minutes**
Canary rollout step progression time	**~1 minute (2 x 30s pauses)**
Deployment rollback time (from failure detection to stable)	**~3–5 minutes**
Redis cache hit rate (repeated identical log queries)	**>90%**
API response time (cached)	**<100ms**
HPA scale-out trigger time (CPU >70%)	**~90–120 seconds**
Docker image size (backend)	**~180MB**
Docker image size (frontend/Nginx)	**~45MB**

7.2 Key Achievements

- Zero-downtime deployments achieved through Argo Rollouts canary strategy on production Kubernetes cluster.
- Shift-left security enforced: Gitleaks, SonarQube, and Trivy integrated at build time, preventing insecure artifacts from reaching ECR.
- Full GitOps implementation: every deployment change is traceable to a specific Git commit, enabling complete audit history and easy rollback.
- Infrastructure-as-code: 100% of Kubernetes resources defined declaratively in Helm charts and YAML manifests, version-controlled in Git.

- Automated rollback: demonstrated successful automated recovery from a failed deployment without manual intervention.
- Scalable application architecture: HPA validated to scale backend pods horizontally under load.
- AI-powered log analysis: LogLens AI successfully analyzes raw application logs and returns structured root-cause analysis and remediation suggestions using Google Gemini.

7.3 Challenges Encountered and Solutions

- Challenge: ArgoCD server TLS termination at ALB caused gRPC connection errors. Solution: Enabled server.insecure mode in ArgoCD config and configured ALB with HTTP backend protocol.
- Challenge: Argo Rollouts dashboard not accessible via ingress. Solution: Configured separate ingress for argo-rollouts namespace with correct service port targeting.
- Challenge: AWS EBS volumes in single AZ causing pod scheduling constraints. Solution: Used topology-aware volume binding and node affinity rules in StorageClass.
- Challenge: SonarQube false positives from Python type annotations. Solution: Configured sonar-project.properties with appropriate exclusion rules for test files and generated code.
- Challenge: Helm values update in pipeline causing merge conflicts on concurrent builds. Solution: Enabled disableConcurrentBuilds() option in Jenkins pipeline options.

Chapter 8: Conclusion & Future Scope

8.1 Conclusion

This project successfully demonstrates the design, implementation, and validation of a comprehensive CI/CD pipeline for Kubernetes-based application deployment using a modern DevOps toolchain. The pipeline integrates Jenkins for build orchestration, ArgoCD for GitOps-based continuous delivery, and Argo Rollouts for progressive canary deployments on Amazon EKS.

The LogLens AI application — an intelligent log analysis system built with FastAPI, React, PostgreSQL, Redis, and Google Gemini AI — serves as a practical, real-world workload that demonstrates all aspects of the pipeline including Docker containerization, Helm packaging, multi-tier service deployment, persistent storage, and AI API integration.

The project proves that with the right toolchain and GitOps principles, software teams can achieve rapid, reliable, and secure software delivery: every code change is automatically built, scanned for security issues, tested, packaged, and deployed to production with zero downtime and automated rollback on failure. This represents the state-of-the-art in cloud-native DevOps engineering as practiced at production scale in modern technology organizations.

8.2 Future Scope

- Prometheus + Grafana Integration: Deploy a full observability stack with Grafana dashboards visualizing application metrics, pipeline metrics, and canary rollout progress.
- Argo Rollouts Metric Analysis: Integrate Prometheus metrics as automated rollout analysis metrics so that Argo Rollouts can automatically abort a canary based on error rate or latency thresholds, without human intervention.
- Multi-Environment Pipeline: Extend the pipeline to support Dev → Staging → Production promotion gates, with approval steps between environments.
- External Secrets Operator: Replace Kubernetes Secrets with ExternalSecrets backed by AWS Secrets Manager or HashiCorp Vault for improved secret lifecycle management.
- Policy as Code: Integrate OPA Gatekeeper or Kyverno to enforce Kubernetes admission policies (e.g., preventing deployments without resource limits or from non-ECR registries).
- DAST (Dynamic Application Security Testing): Add OWASP ZAP or Nuclei scanning stage to the pipeline for runtime security testing of the deployed application.
- Multi-Cloud Support: Abstract the pipeline to support GKE (Google Kubernetes Engine) or AKS (Azure Kubernetes Service) in addition to AWS EKS.
- Blue-Green Deployment Strategy: Implement Argo Rollouts blue-green strategy as an alternative to canary for use cases requiring instant traffic cutover.
- Notification Integration: Add Jenkins and ArgoCD webhook notifications to Slack or Microsoft Teams for real-time deployment status updates.

Chapter 9: References

1. Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.
2. Fowler, M. (2006). Continuous Integration. Retrieved from https://martinfowler.com/articles/continuousIntegration.html
3. Weaveworks. (2017). GitOps: What You Need To Know. Retrieved from https://www.weave.works/blog/gitops-operations-by-pull-request
4. Jenkins Documentation. (2024). Pipeline Syntax. Retrieved from https://www.jenkins.io/doc/book/pipeline/syntax/
5. ArgoCD Official Documentation. (2024). Getting Started. Retrieved from https://argo-cd.readthedocs.io/en/stable/getting_started/
6. Argo Rollouts Documentation. (2024). Canary Deployments. Retrieved from https://argo-rollouts.readthedocs.io/en/stable/features/canary/
7. Kubernetes Documentation. (2024). Horizontal Pod Autoscaling. Retrieved from https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
8. AWS Documentation. (2024). Amazon EKS User Guide. Retrieved from https://docs.aws.amazon.com/eks/latest/userguide/
9. AWS Documentation. (2024). AWS Load Balancer Controller. Retrieved from https://kubernetes-sigs.github.io/aws-load-balancer-controller/
10. Helm Documentation. (2024). Chart Template Guide. Retrieved from https://helm.sh/docs/chart_template_guide/
11. Docker Documentation. (2024). Dockerfile Reference. Retrieved from https://docs.docker.com/reference/dockerfile/
12. SonarQube Documentation. (2024). Jenkins Integration. Retrieved from https://docs.sonarsource.com/sonarqube/latest/
13. Aqua Security. (2024). Trivy Documentation. Retrieved from https://trivy.dev/docs/
14. Gitleaks. (2024). Gitleaks Documentation. Retrieved from https://github.com/gitleaks/gitleaks
15. NIST. (2017). SP 800-190: Application Container Security Guide. National Institute of Standards and Technology.
16. Google AI. (2024). Gemini API Documentation. Retrieved from https://ai.google.dev/docs
17. FastAPI Documentation. (2024). FastAPI. Retrieved from https://fastapi.tiangolo.com/
18. Bitnami Charts. (2024). PostgreSQL Helm Chart. Retrieved from https://github.com/bitnami/charts/tree/main/bitnami/postgresql
19. Bitnami Charts. (2024). Redis Helm Chart. Retrieved from https://github.com/bitnami/charts/tree/main/bitnami/redis
20. AWS Documentation. (2024). Amazon ECR User Guide. Retrieved from https://docs.aws.amazon.com/AmazonECR/latest/userguide/
21. Project GitHub Repository: https://github.com/zoro2024/CU-MCA-final-project/tree/master

Chapter 10: Appendices

Appendix A: Project Repository Structure

```
CU-MCA-final-project/
├── Application/
│   ├── backend/          # FastAPI Python backend
│   │   ├── app/
│   │   │   ├── main.py   # FastAPI routes & startup
│   │   │   ├── ai.py     # Google Gemini integration
│   │   │   ├── db.py     # SQLAlchemy models
│   │   │   ├── cache.py  # Redis caching
│   │   │   ├── config.py # Pydantic settings
│   │   │   └── schemas.py # Pydantic request/response schemas
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   ├── frontend/         # React + Vite frontend
│   │   ├── src/
│   │   │   ├── main.jsx  # Main React component
│   │   │   ├── index.css
│   │   │   └── background3d.js
│   │   ├── Dockerfile
│   │   ├── nginx.conf
│   │   └── package.json
│   ├── helm/
│   │   ├── backend/      # Helm chart for backend
```

```
|   |   |   ├── templates/
|   |   |   |   ├── rollout.yaml
|   |   |   |   ├── service.yaml
|   |   |   |   ├── ingress.yaml
|   |   |   |   ├── hpa.yaml
|   |   |   |   └── job.yaml (DB migration hook)
|   |   |   ├── values.yaml
|   |   |   └── Chart.yaml
|   |   └── frontend/     # Helm chart for frontend
|   └── k8s/              # Raw Kubernetes manifests
├── Jenkinsfile/
|   ├── Backend/Jenkinsfile
|   └── Frontend/Jenkinsfile
└── Tools-installation/
    ├── Argo-cd/          # ArgoCD + Argo Rollouts setup
    ├── Ingress-controller/ # AWS ALB Controller setup
    ├── postgres/         # PostgreSQL Bitnami Helm setup
    ├── redis/            # Redis Bitnami Helm setup
    └── Metric/           # Metrics setup guide
```

Appendix B: Key Configuration Files

B.1 Backend Helm values.yaml (excerpt)

```
image:
  repository: 432995435807.dkr.ecr.ap-south-1.amazonaws.com/cu-final/loglens-backend
  tag: "latest"
secret:
  create: false
```

```
migrate:
  enabled: true
```

B.2 Argo Rollout Canary Strategy (excerpt)

```
strategy:
  canary:
    steps:
      - setWeight: 20
      - pause: { duration: 30s }
      - setWeight: 50
      - pause: { duration: 30s }
      - setWeight: 100
```

B.3 Jenkins Pipeline Image Tag Generation

```
def commitHash = sh(
    script: 'git rev-parse --short HEAD',
    returnStdout: true
).trim()
env.IMAGE_TAG = "${commitHash}-${BUILD_NUMBER}"
```

Appendix C: GitHub Repository

The complete source code, Helm charts, Kubernetes manifests, Jenkinsfiles, and infrastructure setup documentation for this project are available at:

https://github.com/zoro2024/CU-MCA-final-project/tree/master

All README files, PPT and docx file within the repository provide step-by-step instructions for setting up each component of the infrastructure and pipeline.